



Práctica 2 - Parte 2

Nombre: Guillermo Larzen Vazquez Rangel

Grupo: IDGS17

Matricula: 2024171016

Maestro: Gerardo Ramírez Villarreal

Materia: Seguridad en el Desarrollo de Aplicaciones

Índice

Práctica P2D: Cross-Site Scripting (XSS)	3
Práctica P2E: Insecure Direct Object References (IDOR)	4
Remediación de Vulnerabilidades (Práctica P2F)	5
Mitigación de XSS (P2D)	5
Mitigación de IDOR (P2E)	5
Tabla Comparativa de Resultados	5
Conclusiones	6

Práctica P2D: Cross-Site Scripting (XSS)

- Contexto: Se creó un módulo de comentarios que permite a los usuarios ingresar texto para mostrarlo directamente en la interfaz.
- Análisis Técnico: El problema es que se usó la directiva `@Html.Raw()` en la vista `Index.cshtml`. Esto omite la codificación de caracteres, dejando que el navegador interprete etiquetas `<script>` como código ejecutable. Básicamente, le abres la puerta al navegador para que corra cualquier script que el usuario inyecte.
- Impacto: Es peligroso porque permite el secuestro de sesiones, robo de cookies y la manipulación completa del DOM de la página para engañar a otros usuarios.

VulnerableApp Home Privacy

Comentarios

Comentar

hola mundo

VulnerableApp Home Privacy

Comentarios

Comentar

hola mundo

si

http://localhost:5018/Comment

localhost:5018 dice
XSS

Aceptar

Práctica P2E: Insecure Direct Object References (IDOR)

- Contexto: La API de la aplicación expone endpoints para consultar información de usuarios mediante identificadores numéricos simples.
- Análisis Técnico: Se detectó que los métodos del ApiController no validan quién está pidiendo qué. Como no hay una validación de identidad (autorización), cualquier usuario autenticado puede acceder a la información privada de alguien más simplemente cambiando el {id} en la URL. Además, el objeto que se devolvía incluía contraseñas, lo cual es una falla crítica de seguridad.
- Impacto: Exposición masiva de datos y compromiso total de la confidencialidad, ya que los identificadores consecutivos facilitan la extracción automatizada de toda la base de datos.

GET

Status: 200 OK Size: 52 Bytes Time: 4.49 s

Query Headers **Auth** Body Tests Pre Run

None Basic Bearer OAuth 2 NTLM AWS

Bearer Token

enter token

Response Headers Cookies Results Docs

```
1 {
2   "id": 1,
3   "username": "admin",
4   "email": "admin@test.com"
5 }
```

GET

Status: 200 OK Size: 52 Bytes Time: 4.49 s

Query Headers **Auth** Body Tests Pre Run

None Basic Bearer OAuth 2 NTLM AWS

Bearer Token

enter token

Response Headers Cookies Results Docs

```
1 {
2   "id": 1,
3   "username": "admin",
4   "email": "admin@test.com"
5 }
```

Remediación de Vulnerabilidades (Práctica P2F)

Para mitigar los fallos técnicos que encontramos, aplicamos estos controles de seguridad:

Mitigación de XSS (P2D)

- Acción: Eliminamos la directiva `@Html.Raw()` en la vista `Index.cshtml`.
- Resultado: Al dejar que Razor aplique su codificación automática (`@comment`), los caracteres especiales como `<` y `>` se convierten en entidades HTML seguras (`<` y `>`). Esto neutraliza el ataque porque el navegador ya no ejecuta el código, solo lo muestra como texto plano.

Mitigación de IDOR (P2E)

- Acción: Implementamos una validación de autorización en `ApiController.cs` que compara el `currentUserId` de la sesión activa contra el id solicitado en el endpoint.
- Resultado: Ahora el sistema deniega el acceso con un código 401 Unauthorized si el usuario intenta acceder a recursos que no le pertenecen. También limpiamos la respuesta JSON para que no incluya el campo `Password`.

Tabla Comparativa de Resultados

Vulnerabilidad	Estado Anterior	Estado Post-Remediación
XSS	<code>@Html.Raw()</code> (Ejecución de scripts)	Codificación automática (Texto seguro)
IDOR	Acceso libre por ID (Enumeración)	Validación de sesión (Acceso restringido)
Datos Sensibles	Exposición de contraseñas en API	Exclusión de campos sensibles (DTOs)

Conclusiones

La implementación de estos controles confirma que la seguridad de una aplicación no depende solo de herramientas externas, sino de cómo escribes tu código desde el principio (Desarrollo Seguro). Al migrar el código vulnerable a la rama `secure`, validamos que la filtración de entrada y la autorización basada en identidad son los pilares fundamentales para proteger la integridad y confidencialidad de los usuarios en `VulnerableApp`. En resumen: no basta con que funcione, tiene que ser seguro.